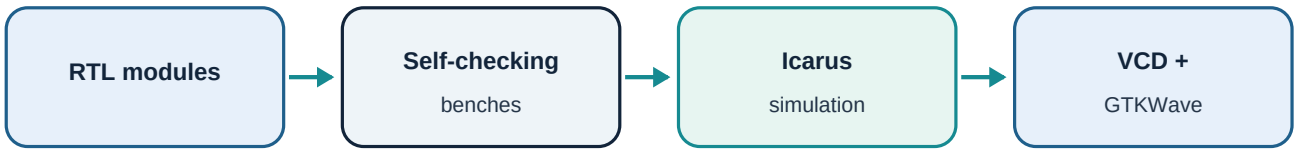
TASK 2 • PROJECT REPORT • UPDATED 8 NOVEMBER 2026

VLSI Design and Semiconductor Engineering

Sequential RTL, counters, parameterized modules, finite state machines, and simulation

Project overview

This project implements and simulates synthesizable Verilog blocks that store and update digital state on clock edges. It includes resettable sequential circuits, five counter designs, reusable parameterized modules, controller FSMs, and a timed traffic-light mini-project.



Project goals

- Design D flip-flops, registers, and shift registers with synchronous and asynchronous reset.
- Implement four-bit up/down, Mod-10, ring, and Johnson counters.
- Build parameterized registers, counters, and shift registers and verify 4-bit and 8-bit configurations.
- Design Moore and Mealy traffic-light FSMs, a vending machine, an elevator, and an autonomous timed traffic-light controller.
- Verify behavior with self-checking testbenches and inspect waveforms in GTKWave.

Development environment

Tool	Recorded environment
Operating system	Ubuntu 26.04.1 LTS under WSL
HDL simulation	Icarus Verilog 12.0 and VVP 12.0
Waveform viewer	GTKWave 3.3.126
Version control	Git 2.53.0

Public repository: github.com/vishnuvamsi396-gif/VLSI-Design-Task-2

DESIGN AND DOCUMENTATION

RTL blocks and controller behavior

Sequential blocks

The sequential RTL provides D flip-flops, parameterized-width registers, and shift registers. Synchronous reset is sampled on the rising clock edge; asynchronous reset clears state as soon as reset is asserted. Sequential state updates use nonblocking assignments.

Counters

Design	Reset	Behavior from reset
4-bit up	0000	0 to 15, then wrap to 0
4-bit down	1111	15 to 0, then wrap to 15
Mod-10	0000	0 to 9, then return to 0
4-bit ring	0001	0001, 0010, 0100, 1000, repeat
4-bit Johnson	0000	Eight-state sequence, then repeat

The full next-state rules and test coverage are in **docs/counter_design.md**.

Parameterized RTL

Reusable register, counter, and shift-register modules use a WIDTH parameter. The self-checking testbench instantiates four-bit and eight-bit variants. Fixed-width modules are used where the exercise specifically asks for four-bit circuits.

Finite state machines and mini-project

Controller	Behavior
Traffic light (Moore)	RED -> GREEN -> YELLOW -> RED; tick advances state.
Traffic light (Mealy)	Same state sequence; emergency input immediately overrides outputs to red.
Vending machine	Accepts 5-unit and 10-unit coins for a 15-unit product; reports a 5-unit overpayment.
Elevator	Three-floor controller accepts a valid target and moves one floor per tick.
Timed traffic-light mini-project	Autonomous Moore FSM with configurable red, green, and yellow durations.

Diagrams and logic tables

The repository includes FSM transition diagrams, circuit-level RTL block diagrams for sequential logic and counters, a timed traffic-light block diagram, a project RTL overview, a clock/reset timing diagram, and truth tables. GitHub previews are indexed in **docs/diagrams/README.md**; truth tables are in **docs/truth_tables.md**.

SIMULATION AND SUBMISSION EVIDENCE

Verification results

The recorded Ubuntu/WSL run used Icarus Verilog 12.0. All seven self-checking testbenches passed. A failure calls $fatal; successful runs write PASS messages and VCD traces.

Verification area	Recorded result and evidence
Sequential logic	Sync/async reset behavior; simulation log and VCD
Counters	Up/down wrap, Mod-10 wrap, ring and Johnson sequences
Parameterized RTL	Register, counter, shift register at 4-bit and 8-bit widths
FSM controllers	Moore/Mealy traffic light, vending machine, elevator
Timed traffic-light mini-project	Reset state and configured state durations
Timing behavior	Sync versus async reset around clock edges

Simulation files

Seven simulator logs are under **simulation/results/**, and seven VCD traces are under **simulation/vcd/**. Run the recorded simulation suite from the repository root with **bash scripts/run_all.sh**.

GTKWave screenshots

Four representative GTKWave PNG captures are in **waveforms/** and embedded in **waveforms/README.md**. The Task 2 assignment requests screenshots but does not specify a required count.

Timing interpretation

Setup time is the minimum interval data must be stable before a capturing edge; hold time is the interval it must remain stable afterward. The timing diagram illustrates clock and reset relationships. Zero-delay RTL simulation demonstrates logical edge behavior but cannot measure physical setup/hold margins; that requires cell/path delays, timing constraints, and static timing analysis.

Submission index

Deliverable	Repository location
RTL and testbenches	rtl/ and testbench/
Simulation logs and VCD traces	simulation/results/ and simulation/vcd/
GTKWave captures	waveforms/
FSM, RTL block, and timing diagrams	docs/diagrams/
Truth tables and counter documentation	docs/truth_tables.md and docs/counter_design.md
README, project PDF, and progress log	README.md, docs/task2_project_report.pdf, progress_log/

The PDF also lists a separate advanced digital-stopwatch exercise; it is outside the required Task 2 submission checklist and is not included in this project.